AIPL: Actor based Intelligent Parallel Language

— 設計から実現 —

第 3 版：採否の再検討と、四つの機構の追加

児玉靖司

法政大学経営学部

yass@hosei.ac.jp

2026 年 8 月 22 日（第 3 版）

概要

本稿は、アクターを単位とする型付き並行言語 AIPL（Actor based Intelligent Parallel Language、処理系名 ABCLc+）の設計と実装を、**何を先行研究から採り、何を落とし、なぜそうしたか**を軸にまとめたものである。第 2 版に対し、採否そのものを見直して**一つを外し、四つを足した**結果を反映している。

源流は二つある。ABCL/1（Yonezawa, Briot, Shibayama, OOPSLA 1986）からは三種のメッセージ送信とオブジェクト内部の逐次性を受け継ぎ、express mode による割り込みと `script` の `where` ガードは落とした。小林直樹の型システム研究の系譜からは効果と「ちょうど一度」の線形性の考え方を採り、交差型による高階モデル検査・多相メソッド・完全な所有権型は落とした。

第 2 版から変わった点は五つである。**(1)** `become` を言語から外した。ABCL/1 に無く、型を守る制限を課すと状態フィールドと分岐で書くのとほぼ同じになり、543 本の実プログラムで一つも使われていなかった。**(2) 失敗を型で表す**機構を入れた。期限つきの待ちで `else` を書かなければ `result`$\langle\tau\rangle$ が返り、期限切れの値を正常値として流せなくなった。**(3) 効果に順序**を入れた。`acquire`/`release` の対を追い、取り残し・二重取得・枝の食い違いを止める。**(4) 返信先を一級の値にした**。ABCL/1 から落とした三つのうち唯一「型の問題」であったものを、線形型で取り戻した —- 委譲が書けて、なお「ちょうど一度」が保たれる。**(5) メッシュ配備**を入れた。アクターのソースを他ノードへ送り、相手先で構文解析・型検査してから実体化する。効果注釈がコードと一緒に運ばれ、受け入れ側の方針と照合される。

併せて**デッドロック**を扱った。調べてみると、AIPL で待ちが返らなくなる経路は**三つ**あり、一つのものとして扱うと答えを誤る —- 循環待ち、返信されない待ち、`select` が来ないメッセージを待つ場合である。二つ目は**閉路が無くても詰まる**ことを実測で確かめた。三つとも手を入れた —- `reply` の全域性、義務レベル、そして `select` への期限の義務づけと送り手の存在検査である。併せて**セッション型**を入れ、やり取りの順序も静的に照合するようにした —- 実行時のプロトコル宣言を型検査の側が読む形で、新しい宣言は要らない。さらに**資源への全体順序**を入れ、逆順の取得によるデッドロックも型で止まるようにした —- こちらも注釈は要らず、**取得の入れ子**がそのまま順序の主張になる。レベルは注釈でも書けるし、**書かなければ推論される**。その結果、**待ちの宛先がクラスとして分かるか、宛先ノードのレベルが宣言されている範囲では、デッドロック自由が型で保証される**。成立の条件と、依然として残る範囲を明記する。

採否の理由はすべて動く最小プログラムで示す。最新仕様の代表的なサンプル 19 本を、コード・出力・メリット・デメリットとともに解説し、そのうち 17 本が**三つの独立な処理系で出力一致**することを確認した（`s11` と `s16` は出力にノード名を含むため自動比較から外し、内容を目視で照合した）。弾かれるべきプログラムは 30 本になった。

目次

1 はじめに 4

2 背景 4
- 2.1 ABCL/1 . . . 4
- 2.2 小林らの型システム研究 . . . 4
- 2.3 他の並行オブジェクト言語 . . . 4

3 ABCL/1 から採ったもの 5
- 3.1 三種のメッセージ送信 . . . 5
- 3.2 オブジェクト内部の逐次性 . . . 5
- 3.3 返信という考え方 . . . 5

4 ABCL/1 から落としたもの 5
- 4.1 express mode による割り込み . . . 6
- 4.2 返信先の一級性 . . . 6
- 4.3 `script` の `where` ガード . . . 6

5 小林研究から採ったもの 7
- 5.1 効果 —- 資源使用解析の素朴版 . . . 7
- 5.2 期限 —- サイズ型の粗い代用 . . . 8
- 5.3 線形性の考え方 —- `reply` は高々一度 . . . 8

6 小林研究から落としたもの 9
- 6.1 交差型による高階モデル検査 . . . 9
- 6.2 多相メソッド . . . 9
- 6.3 完全な所有権の型システム . . . 9
- 6.4 デッドロック自由な型システム（採らず、期限で代用） . . . 9

7 採否の対応表 10

8 `become` —- 入れて、型で縛って、外した 10
- 8.1 無制限だと型が嘘になる . . . 11
- 8.2 型で縛ると、縛った先に残るものが少ない . . . 11
- 8.3 使われていなかった . . . 11

9 第 2 版から足した四つの機構 12
- 9.1 失敗を型で表す —- `result`$\langle\tau\rangle$ . . . 12
- 9.2 効果に順序を入れる —- `acquire` / `release` . . . 12
- 9.3 返信先を一級の値にする —- 線形型 . . . 13
- 9.4 メッシュ配備 —- ソースを送り、相手先で JIT する . . . 14
- 9.5 循環待ちの静的検査 . . . 14

9.6 デッドロックは三つの経路がある . . . 15
9.7 二つは型で防げる . . . 15
9.8 「保証できる」と言うための条件 . . . 16
9.9 経路 3 —- `select` の待ち . . . 17
9.10 やり取りの順序 —- セッション型 . . . 17
9.11 資源の取得順序 —- 資源への全体順序 . . . 18
9.12 ノードをまたぐ待ち . . . 19

10 代表的なサンプルプログラム 20
10.1 s01 —- アクターの基本 . . . 20
10.2 s02 —- 三種の送信 . . . 21
10.3 s03 —- `reply` の規律 . . . 21
10.4 s04 —- 効果 . . . 21
10.5 s05 —- 期限 . . . 21
10.6 s06 —- 実時間側の防壁 . . . 21
10.7 s07 —- `select` . . . 22
10.8 s08 —- 状態機械（`become` を使わずに書く） . . . 22
10.9 s09 —- アクターを引数に渡す . . . 23
10.10 s10 —- 三段のパイプライン . . . 23
10.11 s11 —- メッシュへの配備 . . . 24
10.12 s12 —- 失敗を型で表す . . . 24
10.13 s13 —- 資源の取得と解放 . . . 24
10.14 s14 —- 返信先の委譲 . . . 24
10.15 s15 —- 義務レベル . . . 25
10.16 s18 —- セッション型 . . . 25
10.17 s19 —- 資源への全体順序 . . . 25
10.18 s17 —- `select` の規律 . . . 25
10.19 s16 —- ノードをまたぐ義務レベル . . . 26
10.20 19 本を通して見えること . . . 27

11 三つの処理系と、揃っていることの確かめ方 27
11.1 本稿の 10 本を三実装に通して見つかったもの . . . 27
11.2 現在の状態 . . . 28

12 考察 28
12.1 採否の判断は一貫していたか . . . 28
12.2 「正」の実装という考え方の限界 . . . 29
12.3 型・効果・期限という三点セットの評価 . . . 29
12.4 三つの課題を片付けて分かったこと . . . 29
12.5 機械検証との距離は広がった . . . 30
12.6 残っている課題 . . . 30

13 おわりに 30

1　はじめに

AIPL は二つの目的のために作られている。**分散 AI エージェント**の記述と、**実時間 OS** の記述である。前者は「機外のモデルを呼ぶので、いつ返るか分からない」世界であり、後者は「期限を守れないことが故障である」世界である。両方をアクターとメッセージで書きたい。そして**両者を混ぜてはいけない**。実時間で動くアクターが機外のモデルを同期で待ってはならない。これを規約ではなく型検査で禁じたい —- これが設計を貫く動機である。

言語を一から作るのではなく、二つの先行研究から出発した。ABCL/1 は三種のメッセージ送信を中核に据えた設計であり、この三種はそのまま「待たない／待つ／後で受け取る」という区別に対応する。ただし ABCL/1 は動的型であり、型システムを持たない。その基礎づけを与える試みが、小林直樹らの一連の研究である。

本稿の構成は次のとおりである。§2 で背景を述べ、§3 と §4 で ABCL/1 から採ったもの・落としたものを、§5 と §6 で小林研究から採ったもの・落としたものを、それぞれ動くプログラムとともに述べる。§7 に採否の対応表を置く。§8 は、どちらの源流にも無い `become` の扱いを論じる。§9 が第 2 版から足した四つの機構、§10 が代表的なサンプル 14 本、§11 が三つの処理系、§12 が全体の考察である。

本稿のプログラムはすべて実際に処理系へ通した。出力は貼り付けたものではなく、実行して得たものである。

2　背景

2.1　ABCL/1

ABCL/1 は並行オブジェクトを計算の単位とする言語である。オブジェクトは自分のメールボックスを持ち、到着したメッセージを一つずつ処理する。この「一度に一つ」がオブジェクト内部の逐次性を保証し、それゆえフィールドへの排他制御を書かなくてよい。

中核は四つ —- 三種のメッセージ送信、`script` によるパターン受信（`where` ガードつき）、express mode による割り込み、返信先（reply destination）の一級性である。

2.2　小林らの型システム研究

小林と米澤の一連の研究は、線形論理を土台として並行オブジェクトの型理論的基礎を与えた。重要なのは、AIPL が直面する問題の多くがすでにこの系譜で扱われていることである—- 線形型（「ちょうど一度」）、デッドロック自由な型システム、π 計算の汎用型システム、資源使用解析、高階モデル検査と交差型の等価性、述語抽象と CEGAR、精製型・サイズ型による計算量。

2.3　他の並行オブジェクト言語

比較の座標として次を置く。ABCL/f は同じ ABCL 系で型を持つ先行例で、**future だけを残して型を付ける**という判断をとった。Erlang/OTP は `gen_statem` により状態遷移を書くが、コールバックが受け取るメッセージ集合は固定である。Akka は classic では `context.become` が無型であったが、**Akka Typed で `Behavior[T]` の `T` を固定する形に作り直した**。この最後の一点は §8 で効いてくる。

3　ABCL/1 から採ったもの

3.1　三種のメッセージ送信

最も本質的な継承である。

AIPL の書き方	ABCL/1	意味	式か文か
`send t.m(a);`	past	送って待たない	文
`now t.m(a) timeout` n `else` e	now	送って返信を待つ	式
`future t.m(a)` / `await f ...`	future	引換券を受け取る	式

```
class Calc {
  method twice(x) : int { reply(x * 2); }
  method slow(x)  : int { reply(x + 100); }
}
var c = new Calc();
send c.twice(1);                                          // 待たない
print("now    = " ++ (now c.twice(21) timeout 100 else 0)); // 待つ
var f = future c.slow(5);                                 // 引換券
print("future = " ++ (await f timeout 100 else 0));       // あとで受け取る
```

```
now    = 42
future = 105
```

■なぜ採ったか。 この三分割は、そのまま AIPL が必要とする区別である。実時間側は「待たない」を多用し、エージェント側は「後で受け取る」を使う。ABCL/f は future だけを残したが、AIPL は三種すべてを残した。そのぶん型付けは面倒になる —- `send` は返信を受け取らないので戻り値型を要求せず、`now` は式なのでその場に戻り値型が要り、`future` は τ を返すメソッドに `future`$\langle\tau\rangle$ を与える。この面倒は、三種の区別が設計上必要だという判断に対する代価である。

3.2　オブジェクト内部の逐次性

アクターはメッセージを一つずつ処理する。フィールドへの代入に排他制御が要らないのはこのためで、§5 で述べる効果 `mut` が意味を持つのもこの前提のうえである。

3.3　返信という考え方

値は `return` ではなく `reply` で返す。これは ABCL/1 の返信をそのまま受けたものだが、静的型を載せるうえでは厄介な選択である。`return` なら関数の型が戻り値型を決めるが、`reply` は本体のどこにでも書ける文だからである。AIPL はメソッドごとに戻り値型を表す型変数 ρ を置き、本体の各 `reply(e)` と単一化することで推論する。

4　ABCL/1 から落としたもの

三つとも「入れられない」のではなく、**入れると証明の形が変わる**という理由で落とした。

4.1　express mode による割り込み

ABCL/1 は ordinary モードとは別の待ち行列を持ち、express メッセージは実行中の処理を中断して割り込む。

■落とした理由。 §3 の逐次性が破れる。「メソッド本体の実行が最後まで途切れない」という前提のうえに、フィールドの不変条件も、効果の集計も、型健全性の証明も立っている。割り込みを入れると、本体の途中で状態が観測されうるので、これらをすべて「割り込み点で成り立つ不変条件」に書き直す必要がある。

■落とすと何が書けなくなるか。 中止・優先処理・監視である。AIPL では `select` と期限で近いことを書くが、**実行中の処理を止めることはできない**。できるのは「次に何を受けるか」を選ぶことだけである。

4.2　返信先の一級性

ABCL/1 では *now* 型送信で暗黙に作られる返信先が値であり、他へ渡せる。つまり「A が受けた依頼に B が代わりに答える」委譲が書ける。

■落とした理由。「ちょうど一度だけ返信する」を保てないからである。AIPL は `reply` の回数を**構文の走査**で数えている —- 本体に現れる `reply` の上界を数え、2 以上なら拒否する。これは返信先が暗黙である限り成立するが、値として渡せるようにした途端、渡された先で何回返されるかは走査では分からない。線形型を入れれば解けるが、それは §6 の話になる。

■副産物。 この判断には思わぬ効果があった。`self` と `sender` は**送信の宛先としてしか書けず、値ではない**。

```
send self.h(1);          -> OK
send e.take(self, x);    -> PARSE_ERROR
```

そのため、二者が互いに `now` で待ち合う循環を、現在の構文では素直に組み立てられない。デッドロック自由性を型で保証しているわけではないが、**一級性を落としたことが、待ち合いを作りにくくもしている**。

4.3　`script` の `where` ガード

ABCL/1 は受信のパターンに `where` 節を書け、「在庫があるときだけ購入メッセージを受理する」といった記述ができた。AIPL の `select` はパターンをメソッド名と引数の並びに限り、`where` を持たない。

■落とした理由。 進行定理に枝が一本増えるからである。どのガードも成立せず受理できない構成が現れうるので、「終状態」「簡約できる」に加えて「受理できるメッセージが無い」を扱う必要が出る。さらにガードの純粋性という新しい要求も生じる（ガードが状態を変えてはならない）。

■落とすと書き方がどう変わるか。 受理してから本体で分岐する。

```
class Shop {
  var stock = 2;
  method buy(n) : string !{mut} {
    if (stock >= n) { stock = stock - n; reply("ok, left=" ++ stock); }
```

```
    else { reply("sold out"); }
  }
}
var s = new Shop();
print(now s.buy(1) timeout 200 else "?");
print(now s.buy(5) timeout 200 else "?");
```

```
ok, left=1
sold out
```

違いは**受理してしまう**点にある。`where` なら「受理しない＝送り主は待ち続ける（あるいは他が受理する）」であったのが、AIPL では「受理して、断りの返信をする」になる。在庫が戻るまで待たせたい場合、その待ちはプログラマが書くことになる。

5　小林研究から採ったもの

5.1　効果 —- 資源使用解析の素朴版

メソッドは自分が世界に及ぼす作用を宣言できる。効果は 8 種類の固定語彙である。

`mut`	自分の状態を変える	`net`	ネットワークに出る
`time`	時間に依存する	`ai`	言語モデルを呼ぶ
`io`	入出力	`fs`	ファイルを触る
`mem`	動的に割り付ける	`log`	ログを出す

```
class Store {
  var n = 0;
  method bump()  : int !{mut}  { n = n + 1; reply(n); }   // 状態を変える
  method peek()  : int !{}     { reply(n); }              // 純粋
  method show()  : unit !{log} { print("n=" ++ n); }
}
```

書かなければ本体から推論し、書いた場合は本体から推論した集合と照合する。足りなければ型エラーになる。

■採った理由。　これが §1 の動機に対する直接の答だからである。実時間側のアクターに `!{time, log}` と書いておけば、そこから `ai` や `net` を持つメソッドを**同期で**呼ぶことが型エラーになる。

```
[Type error] method Ctrl.viaNow declares {log, time}
             but its body has {ai, log, net} (missing {ai, net})
```

小林の資源使用解析に比べれば素朴である —- 効果は**順序を持たない集合**で、「開いたら閉じる」のような性質は書けない。そこは今後の課題として残っている（§12）。

■規律の細部。　効果を引き継ぐのは**待つ**呼び出しだけである。`send` は送って待たないので引き継がない。この判断は明示的なもので、逆に `send` も引き継ぐようにして手元の `.aipl` 543 本を通してみたところ、落ちたのは**この規律を説明している例題ただ一本**であった。

5.2 期限 —- サイズ型の粗い代用

now と await には期限を書ける。

```
class Slow {
  method fast(x) : int { reply(x); }
  method never(x) : int { while 1 > 0 do { } reply(x); }   // 返ってこない
}
var s = new Slow();
print("fast  = " ++ (now s.fast(42) timeout 200 else -1));
print("never = " ++ (now s.never(7) timeout 200 else -1));
```

```
fast  = 42
never = -1
```

■採った理由と、その位置づけ。 小林のサイズ型・精製型は応答時間の上界を**型で**述べる。AIPL の期限はそこまで行かず、実行時に打ち切るだけである。しかし効果は非常に大きい —- 期限のない待ちを構文から外すと、待ちが「残り時間を減らす簡約」として進むので、進行定理から「未解決 future の待ちでブロックする」枝が消える。系としてデッドロック自由が言える。

代償は明確である。まず停止性は言えない。そして**デッドロックが消えるのではなく、有限時間の失敗に変わる**。

```
class Never  { method wait_forever(x) : int !{} { while 1 > 0 do { } reply(x); } }
class Caller {
  var n = new Never();
  method go(x) : int !{} { reply(now n.wait_forever(x) timeout 200 else -1); }
}
var c = new Caller();
print("go = " ++ (now c.go(5) timeout 900 else -999));
```

```
go = -1
```

詰まらない。しかし正しくもない。−1 が返るだけである。

5.3 線形性の考え方 —- reply は高々一度

線形型そのものは採っていないが、「ちょうど一度だけ使う」という規律は reply に持ち込んでいる。

- **線形性**：一つのメッセージに対する reply は高々一度。
- **被覆**：戻り値型を宣言したら、すべての実行経路で reply に到達する。

```
[Type error] method f may reply more than once on some path;
             a method must reply at most once
[Type error] method f is declared to reply with int,
             but some execution path does not reply
```

ただし実現は**構文の走査**であって型ではない。そのため §4 で述べたとおり、返信先を値にした瞬間に無力になる。

6　小林研究から落としたもの

6.1　交差型による高階モデル検査

交差型による型付けと μ 計算のモデル検査の等価性は、検証器を自作せずに借りるための強力な道具である。

■落とした理由。 型推論が一般に決定不能であり、AIPL の対象（アクターと通信）に対して必要な表現力を超えている。加えて、反例が AIPL の言葉で出てこない —- 検査器が返すのは高階再帰スキームの反例であって、「どのアクターがどのメッセージで詰まるか」ではない。利用者に見せられない反例は、検証としては使えても道具としては使えない。

6.2　多相メソッド

メソッドは単相である。同じメソッドを二つの型で使うことはできない。

```
class Id { method id(x) : any { reply(x); } }
var i = new Id();
print(now i.id(1) timeout 100 else 0);
print(now i.id("s") timeout 100 else "?");
```

```
[Type error] line 3, col 7: type mismatch
```

■落とした理由。 戻り値型を表す ρ を型スキームに含めると、**呼び出しごとに ρ が新しくなる**。すると「このメソッドはどの型を返すか」という制約が呼び出し側へ伝わらず、`reply` の地点と結ばれなくなる。ρ はクラスのメソッド表に属する量であって、多相化の対象ではない。単相のままにしておくのが、`reply` からの推論と型健全性の証明の双方に対して整合的である。

■代価。 上の `Id` のような汎用のアクターが書けない。実際には `any` を使って型検査を素通りさせることになるが、それは検査を放棄しているだけである。

6.3　完全な所有権の型システム

Rust 相当の借用検査は作らない。CHC への還元に必要な範囲（別名が無いこと）は、アクターがメッセージでしか相互作用しないという性質から既に成立しているためである。

6.4　デッドロック自由な型システム（採らず、期限で代用）

通信の依存関係に順序を入れて循環待ちを型で排除する手法は、AIPL の課題にそのまま噛み合う。しかし現状は採っていない。

■採らなかった理由。 義務レベルの推論が利用者に説明しにくいこと、そして期限による代用で**工学的には**詰まらなくなるためである。ただし §5 で示したとおり、代用は保証ではない。将来は「明示宣言のみ」の形から始めるのが現実的だと考えている。

7　採否の対応表

機構	採否	理由	AIPL での姿
ABCL/1 から			
三種の送信	採る	「待たない／待つ／後で」は設計上必要な区別	`send` / `now` / `future`+`await`
内部の逐次性	採る	排他制御を書かずに済む。証明の土台	一度に一つ処理
`reply` による返信	採る	源流の考え方。型付けは ρ で解決	`reply(e)`、戻り値型は推論または注釈
express mode	落とす	逐次性が破れ、証明を全面的に書き直す必要	無し。`select` ＋期限で近いことのみ
返信先の一級性	**採り直す**	線形型を入れれば「ちょうど一度」を守れる	`replyto` / `answer` / 型注釈 `reply`（第 3 版）
`where` ガード	落とす	進行定理に枝が増え、ガードの純粋性が要る	`select` は名前と引数のみ。本体で分岐
小林らの研究から			
資源使用解析	採る	実時間側とエージェント側を分ける唯一の手段	8 種の効果集合＋ `acquire`/`release` の順序（第 3 版）
サイズ型・精製型	採る（粗い代用）	上界を型で言う代わりに実行時に打ち切る	`timeout` n `else` e、および失敗を型に出す `result`$\langle\tau\rangle$（第 3 版）
線形型の考え方	採る（構文で）	「ちょうど一度」は返信の要	`reply` の線形性・被覆検査
線形型（型として）	**採る**	返信先を一級にするために必須になった	返信先の線形性検査（第 3 版）
デッドロック自由な型	落とす	義務レベルの推論が説明しにくい	期限＋循環待ちの静的検査で代用。**保証ではない**（第 3 版）
交差型モデル検査	落とす	推論が決定不能。反例が言語の言葉で出ない	無し
多相メソッド	落とす	ρ をスキームに含めると reply と結べない	単相のみ
完全な所有権型	落とす	別名の無さは既に成立	無し
どちらの源流にも無いもの			
`become`	**外す**	型で縛ると状態フィールドとほぼ同じ。543 本で実使用ゼロ	無し。状態フィールドと分岐で書く（第 3 版）
メッシュ配備	**採る**	複数 Xinu ノードでの実行に要る。効果注釈が入国審査になる	`source_of` / `node_allow` / `deploy`（第 3 版）

8　become —- 入れて、型で縛って、外した

`become` は ABCL/1 には無い。ABCL/1 は状態変数と `script` の制約で同じことを表現していた。AIPL の `become` は Hewitt 系アクターモデル由来である。第 2 版はこれを「型で縛って残す」としたが、

第 3 版で外した。その経緯を書いておく。

8.1 無制限だと型が嘘になる

```
class Closed2 {
  method state() : unit !{log} { print("closed"); }
  method open()  : unit !{mut, mem, log} { become Opened2(); }
}
class Opened2 {
  method opened() : unit !{log} { print("opened"); }   // state も open も無い
}
var d = new Closed2();
send d.open();
send d.state();        // become 後には存在しないメソッド
```

これは**型検査を通った**。d の静的な型は actor(Closed2) であり、そこに state はある。しかし実行時には無い。つまり become は、アクター型の区別という規律を単独で無効にしていた。

8.2 型で縛ると、縛った先に残るものが少ない

型のある設定では一貫して制限が課されている。Akka は classic の無型 context.become を、Akka Typed で Behavior[T] の T を固定する形に作り直した。Erlang/OTP の gen_statem も、状態は変わってもメッセージ集合は固定である。

そこで一度、**置換先は置換前が受け取れたメッセージをすべて受け取れること**を型検査で要求した。

```
[Type error] become Opened2: Opened2 does not accept state, open,
             which Closed2 accepts
```

これは正しく効いた。しかし満たす書き方は、**すべての状態がすべてのメッセージを受けること**になる。そこまで来ると、状態フィールドと本体の分岐で書くのとほとんど変わらない。

8.3 使われていなかった

決め手は使用実態であった。手元の .aipl **543 本**を調べたところ、become を使っていたのは **4 ファイルだけ**で、しかもすべて become 自身のテストであった（become.aipl と bbecome.aipl が二つのリポジトリに重複）。実アプリケーション —- 避難支援 MANET、6 軸アーム、Xinu、進化計算 —- では**一つも使われていなかった**。なお「使用」を grep で数えると 11 ファイルになるが、残りはコメント中の英単語 *become* であった。

所見 1. 判断の順序を記しておく。まず**型を壊すこと**が分かり、次に**型で縛れること**が分かり、最後に**縛った先に残る価値が小さいこと**と**誰も使っていないこと**が分かった。最初の二つだけなら「縛って残す」で終わっていた。外す判断ができたのは、**使用実態を数えたから**である。機能の要否は、設計の議論だけでは決まらない。

外したあとの書き方は §10 の s08 に示す。メッシュでの「役割の付け替え」は become ではなく、§9.4 の配備で表す。

9　第 2 版から足した四つの機構

9.1　失敗を型で表す —- result⟨τ⟩

第 2 版の考察で、現状の最も弱い箇所として「期限切れの else の値が正常値と同じ型である」ことを挙げた。−1 が返ることと、正しく計算できたことを型が区別しない。

■仕様。 期限つきの待ちで else を書かなければ、式の型は result⟨τ⟩ になる。

書き方	型
now t.m(a) timeout 100 else 0	τ（従来どおり）
now t.m(a) timeout 100	result⟨τ⟩（新）

この形は**従来は構文エラーだった**ので、既存プログラムの意味は変わらない。取り出しは is_ok / timed_out / value で行う。

```
var a = now s.fast(42) timeout 200;      // result<int>
if (is_ok(a)) { print("fast ok " ++ value(a, -1)); }
else          { print("fast timed out"); }
```

result⟨τ⟩ は演算子に渡せない。

```
[Type error] no overload of + matches (result int, int)
```

■効いていること。「この値は本当に計算されたのか」をプログラムが問える。確かめずに流すことができない。543 本を通した結果、影響を受けたファイルは 0 本であった。

9.2　効果に順序を入れる —- acquire / release

第 2 版で述べたとおり、効果は**順序を持たない集合**であり、「開いたら閉じる」「取得したら解放する」が書けなかった。小林の資源使用解析のいちばん素朴な形を入れた。

```
class Bus {
  var n = 0;
  method read(addr) : int !{mut} {
    acquire("i2c");
    n = n + 1;
    var v = addr * 2;
    release("i2c");           // これを消すと型エラーになる
    reply(v);
  }
}
```

規律は三つで、いずれもメソッド本体を経路に沿って追って検査する。

1. メソッドを抜けるとき、持っている資源が残っていてはならない
2. 持っていない資源を release してはならない
3. 二重に acquire してはならない

`if` の二つの枝は同じ持ち物で合流し、`while` の本体は持ち物を変えてはならない。

```
[Type error] method Dev4.use returns while still holding i2c
[Type error] resource i2c is acquired twice
[Type error] the two branches disagree on which resources are held (i2c)
```

■限界。 メソッド内で閉じた検査であり、メソッドをまたぐ受け渡しは見ない。また資源に**全体の順序を課していない**ので、取得順序の食い違いによるデッドロックは防げない（§9.5）。

9.3 返信先を一級の値にする —- 線形型

§4 で述べたとおり、ABCL/1 から落とした三つのうち**唯一「型の問題」であった**のが返信先の一級性である。落とした理由は「ちょうど一度だけ返信する」を構文の走査では守れないことであった。線形型で取り戻した。

■仕様。 `replyto` が、いま処理しているメッセージの返信先を値として返す。型は `reply`$\langle\rho\rangle$（ρ はそのメソッドの戻り値型）。`answer(r, v)` で返信する（`reply(v)` はこの糖衣にあたる）。引数の型注釈 `reply` で、他のアクターへ渡せる。

```
class Backend {
  method handle(r: reply, x) : unit !{} {
    answer(r, x * 2);            // Frontend の依頼者へ直接返す
  }
}
class Frontend {
  var b = new Backend();
  method ask(x) : int !{} {
    send b.handle(replyto, x);   // 返信先を渡して義務を移す。自分は reply しない
  }
}
var f = new Frontend();
print("ans = " ++ (now f.ask(21) timeout 500 else -1));
```

```
ans = 42
```

委譲が書けて、なお「ちょうど一度」が保たれる。これが ABCL/1 に無かったものである。

■線形性の規律。 `replyto` で義務が生まれ、`answer` するか送信の引数に渡すと義務が消える（相手へ移る）。同じ返信先を二度使ってはならず、メソッドを抜けるとき義務が残っていてはならず、`if` の二つの枝は同じ状態で合流する。

```
[Type error] method B19.h returns without answering r
[Type error] reply destination r is used twice
[Type error] the two branches disagree on which reply destinations are owed
```

注意 2. 実装で要点になったのは**状態の持ち方**である。「まだ答えていない」集合から消すだけだと、**二度渡しがただの変数参照に見えて素通りする**。$(\mathrm{owed}, \mathrm{spent})$ の対にして初めて捕まった。線形型は「使ったら消す」ではなく「使ったことを憶える」ことで成り立つ。

9.4　メッシュ配備 —- ソースを送り、相手先で JIT する

実行基盤として複数の Xinu ノードをメッシュで繋ぐ構想がある。そこではアクターのソースをネットワーク越しに送り、相手先で組み立てて動かす記述が要る。

```
node_allow("rt0",   "mut, log, time");        // 実時間ノードが許す効果
node_allow("edge0", "mut, log, ai, net");

var h = deploy("rt0", "Servo", "servo1");     // ソースを送り、相手先で JIT
send remote("rt0", "servo1").step(3);
```

```
[deploy] Servo -> rt0/servo1 (compiled at destination)
pos=3
[Top-level CallStmt error] deploy: node rt0 does not accept effect(s) {ai, net}
                           required by Planner
```

要点は、**効果注釈がコードと一緒に運ばれ、受け入れ側の方針と照合される**ことである。実時間ノードに `ai` や `net` を持つコードを配ろうとすると、配備そのものが失敗する。§5 の効果が、単一ノード内の規律から**ノード間の入国審査**になる。

■送る荷物。 クラスを抽象構文木から組み立て直すのではなく、**ソースの原文**を送る。組み立て直すと `select` の中身が落ちるためである。輸送は現状ひとつのプロセス内で模しているが、言語から見える形 —- 何を送り、相手が何を検査するか —- は実機と同じである。

9.5　循環待ちの静的検査

四つの機構を入れる過程で、副産物が一つ出た。効果の伝播に使っている辺（待つ側 → 待たれる側）が、**そのまま待ちグラフ**であった。閉路検出はほぼ費用ゼロである。

```
[Type error] circular wait: Peer2#bounce;
             a now/await cycle deadlocks unless a deadline breaks it
```

543 本すべてで閉路が 0 件だったので、既定をエラーにした（`AIOS_LAX_WAIT=1` で警告に降格）。

■ただし、これだけではデッドロック自由にならない。 次の 10 行は**型検査を通り**、実行すると詰まる。

```
class Peer {
  method tick(n) : int !{} { reply(n); }
  method bounce(p: Peer, n) : int !{} {
    if (n > 0) { reply(now p.bounce(p, n - 1) timeout 300 else -1); }
    else { reply(0); }
  }
}
var p = new Peer();
print("bounce = " ++ (now p.bounce(p, 3) timeout 1500 else -999));
```

```
bounce = -1
```

アクターは一度に一つしかメッセージを処理しないので、自分への `now` は原理的に返らない。-1 は**期限が救っただけ**であって、型は何も止めていない（この例は閉路検査が拾うが、検査はメソッド単位で保守的かつ不完全であり、遠隔配備先は見えない）。

9.6　デッドロックは三つの経路がある

「型でデッドロック自由を保証できるか」を調べたところ、**デッドロックを一つのものとして扱うと答えを誤る**ことが分かった。AIPL で待ちが返らなくなる経路は三つあり、要る道具が違う。

■経路 1 ―― 循環待ち。 上の `Peer#bounce` である。`now` の輪。

■経路 2 ―― 返信されない待ち。 閉路が無くても詰まる。これは実測で見つけた。

```
class Silent2 { method f(x) : unit !{log} { print("ignored"); } }
class Caller4 {
  var s = new Silent2();
  method go(x) : unit !{log} {
    var r = now s.f(x) timeout 200;      // result<unit>
    print("after " ++ is_ok(r));
  }
}
```

```
ignored
after false
```

待ちグラフは非巡回である。それでも返信は永久に来ない。`is_ok` が `false` なのは**期限が救っただけ**である。

■経路 3 ―― `select` が来ないメッセージを待つ。 `select { case j(k) -> ... }` を期限なしで書くと型検査は通る。これも閉路ではない。

9.7　二つは型で防げる

経路	型で防げるか	要る道具
1. 循環待ち	**はい**	義務レベル（小林 1997/2000/2006）
2. 返信されない待ち	**はい**	`reply` の全域性
3. `select` の待ち	部分的に	期限の義務づけ＋送り手の存在検査
3′. やり取りの順序	**はい**	セッション型（実行時の宣言を静的に検査）
4. 取得順序の食い違い	**はい**	資源への全体順序（入れ子から辺を集めて閉路を見る）

■経路 2 ―― `reply` の全域性。 戻り値型を宣言したメソッドには全経路での `reply` を課していたが、`unit` を返すメソッドが抜け穴になっていた。**`now`/`await` で待たれるメソッドに限って**、戻り値型に関わらず課すことにした。待たれるメソッドの集合は、効果の伝播に使う辺からそのまま取れる。

```
[Type error] method Silent2.f is waited on by now/await
             but does not reply on every path
```

563 本を通した結果、**影響を受けたファイルは 0 本**であった。

■経路 1 ―― 義務レベル。 各メソッドにレベルを付け、`now`/`await` は**厳密に大きいレベルにしか向かえない**とする。全メソッドに付けば、待ちのグラフは構成的に非巡回になる。

```
class Source9 { method get(i) : int @ 3 !{} { reply(i * 10); } }
```

```
class Scale9  {
  var s = new Source9();
  method at(i) : int @ 2 !{} { reply((now s.get(i) timeout 100 else 0) + 1); }
}
class Sink9 {
  var c = new Scale9();
  method run(n) : int @ 1 !{log} { ... now c.at(n) ... }
}
```

待ちは $1 \to 2 \to 3$ と一方向に流れる。逆流も同レベルも弾かれる。

```
[Type error] obligation level: B9#g (@2) waits on A9#f (@2);
             a wait must go to a strictly higher level
[Type error] obligation level: B10#g (@5) waits on A10#f (@1); ...
```

■注釈は推論できる。 [25] の助言に従い明示宣言から始めたが、そこには弱点があった —- **片方にしか注釈が無い辺は検査されない**。注釈を付け忘れた組が素通りする。効果注釈と同じ弱点である。

そこで、注釈が無いメソッドには**レベルを推論する**ようにした。規則は一つで、待ちの辺 (a, b) について $\mathrm{level}(b) \leq \mathrm{level}(a)$ なら b を押し上げる、を不動点まで繰り返す。明示注釈は固定点であり、**押し上げが要るのに動かせなければ、そこが矛盾**である。

```
$ AIOS_SHOW_LEVELS=1 tc s10_pipeline.aipl
[level] Sink#run                    @0
[level] Scale#at                    @1
[level] Source#get                  @2
OK
```

注釈をひとつも書いていないプログラムに、待ちの構造がそのまま出てくる。混在しても誤検出しない —- `Mid#at @5`（declared）に対して `Src#get` は `@6` と推論される。矛盾する注釈は捕まる。

```
[Type error] obligation level: Mid2#at (@9) waits on Src2#get (@1);
             a wait must go to a strictly higher level
```

閉路があると不動点に達しないので、閉路検査を先に走らせて具体的な診断を出す。これも既存コードへの影響は 0 本である。

所見 3. 推論を入れて分かったことがある。**レベル推論は、閉路検査に「証拠」を与えたものである**。閉路が無いことと、レベルの割り当てが存在することは同じである。違うのは、閉路検査が「無い」としか言わないのに対し、推論は**どういう順序で待っているか**を数として見せることである。そして混在を許すことで、**人が書いた規律**（明示注釈）と**コードが持つ構造**（推論）を突き合わせられるようになった。

9.8 「保証できる」と言うための条件

二つを入れた現在でも、**全言語について**デッドロック自由が言えるわけではない。条件を明記しておく。

- **引数で受け取ったアクターは、注釈があれば覆われている**。`method g(s: Svc1, x)` と書けば `Use1#g @0` → `Svc1#f @1` と辺が張られる。注釈が無ければ、そもそも「`target is not actor`」で弾かれる。つまりレベルを**アクター型に載せる必要は無かった**。
- **精度が落ちる**。レベルはメソッド単位なので、同じクラスの別インスタンス同士（親が子を待つ木構造など）も一律に弾かれる。緩めるにはレベル変数が要る。

- **ノードをまたぐ待ちは、ノード単位で階層化する**（下記）。
- **停止性・飢餓は別問題**。`while 1>0 do {}` がメールボックスを塞ぐのはレベルでは止まらない。
- 経路 3（`select`）は**部分的**である。送信が動的（`sender` 経由・実行時に決まる宛先）だと見えない。完全には、セッション型が要る。

9.9 経路 3 —- `select` の待ち

三つ目は**閉路でも返信漏れでもない**。誰も送らないだけで詰まる。セッション型がなければ完全には扱えないが、実用上ほとんどを占める二つの形は押さえられる。

■規律 1 —- 期限を書く。 期限の無い `select` は永久に待ちうる。`now/await` と同じ扱いにした —- 既定は警告、`AIOS_STRICT_DEADLINE=1` でエラーに昇格する。永久の待ちが有限時間の失敗に変わる。

■規律 2 —- 送り手が居るか。 `select` の `case` が待つメッセージを、プログラムの中で誰かが送っているかを見る。送り手がどこにも無ければ、その `case` は永久に発火しない。

```
[warn] select waits for W2.ghost but nothing in this program sends it
```

■外部の送り手。 この検査は最初、563 本で 2 件の警告を出した。調べると**どちらも `web_expose` で外へ公開している例題**で、送り手は HTTP の向こう側にいた。`web_expose` / `web_listen` / `deploy` / `remote` を使うプログラムでは、この検査は当てにならないので行わない。結果、563 本での警告は **0 件**になった。

所見 4.「誰も送らない」を静的に言うには、**プログラムの外側を数えなければならない**。効果や返信は言語の中で閉じているが、メッセージの送り手は Web の向こうにも、別のノードにもいる。この検査が既定で警告なのはそのためである —- **言語が世界の全部を見ているわけではない**という事実が、そのまま検査の強さの上限になる。

9.10 やり取りの順序 —- セッション型

経路 3 には、もう一つの側面がある —- **順序**である。「開いてから読み、最後に閉じる」という約束は、期限でも送り手の存在でも表せない。

■既にあったものを持ち上げた。 AIPL には**実行時**のセッションプロトコルが既にあった —- `protocol_define` / `protocol_start` / `protocol_end` で、Coq でも `send_ok_unique`、`send_ok_advances_one`、`wrong_label_rejected` などが検証されている。新しく作ったのではなく、**同じ宣言を型検査の側が読む**ようにした。

```
protocol_define("Pipeline", "f.get -> r.scale -> st.put");
var sid = protocol_start("Pipeline");

var a = now f.get(4)    timeout 200 else 0;
var b = now r.scale(a)  timeout 200 else 0;
var c = now st.put(b)   timeout 200 else 0;
protocol_end(sid);
```

```
pipeline = 41
```

順序を取り違えると、走らせる前に出る。

```
[Type error] protocol P29: expected b.step2 but the program sends c.step3 here
```

■誤検出を二度直した。 最初の実装は 563 本のうち **7 本**を落とした。どれもセッションがアクターをまたぐ例で、手順の続きは受け手のアクターの中で進んでいた。そこで、手順が全部この並びに現れる場合だけ「やり残し」を誤りとし、そうでなければ既定では黙ることにした（AIOS_STRICT_PROTOCOL=1 で警告)。

それでも 1 本残った。手順が "main_thread.run" なのにプログラムは "main" へ送る、という**綴りの違い**で順序検査が反応していた。手順に含まれない宛先への送信は、このセッションとは無関係なので見ない。

■結果。 563 本で新たに落ちるのは **1 本だけ**である —- session_protocol_violation.aipl。名前のとおり、意図的な違反の例題であった。**実行して初めて分かっていたものが、型検査で出るようになった。**

所見 5. この節の作業は、新しい理論を持ち込む話ではなかった。**実行時にできていたことを、静的にもできるようにした**だけである。そして二度の誤検出は、どちらも「静的に見えるものと、実行時に起きることの差」から出た —- セッションはアクターをまたぐし、宛先の名前は宣言と食い違いうる。§9.9 の所見と同じ形である —- **言語が世界の全部を見ているわけではない。**

9.11　資源の取得順序 —- 資源への全体順序

デッドロックには、もう一本の経路がある。これは待ちの閉路でも、返されない返信でもない —- **二つのアクターが、同じ二つの資源を逆の順序で取る**という形である。

§9.2 の対の検査は、この形を素通しする。どちらのメソッドも、取ったものは全部返しているからである。

```
class Writer {
  method run() : int !{mut} {
    acquire("db"); acquire("cache");
    release("cache"); release("db"); reply(1);
  }
}
class Reader {
  method run() : int !{mut} {
    acquire("cache"); acquire("db");     // 逆順
    release("db"); release("cache"); reply(2);
  }
}
```

■入れ子が順序を語っている。 新しい注釈は要らなかった。r **を持っているあいだに** s **を取った**という事実が、そのまま $r < s$ という辺である。プログラム全体からこの辺を集め、**閉路が無いこと**を見る。

```
[Type error] resource order: cache -> db -> cache is circular;
  cache before db (in Reader.run); db before cache (in Writer.run).
  Acquiring in opposite orders deadlocks
```

閉路の各辺が**どこで生まれたか**を持っておくと、「逆順に取っている二か所」がそのまま出る。デッドロックの報告としては、これがいちばん役に立つ形である。

■閉路が無い＝全体順序が作れる。 これは §9.6 の義務レベルと同じ構造をしている。閉路が無いなら位相順序が取れ、その高さがそのまま「どれを先に取るか」の番号になる。レベルはここでも**証人**である（AIOS_SHOW_LEVELS=1）。

```
[resource] bus                  @0
[resource] dev                  @1
```

■書きたい人のために。 推論に任せず resource_order("bus -> dev") と書くこともできる。宣言は辺として同じ表に入るだけなので、宣言と実際の取得が食い違えば、それも閉路として出る ―- 違反を見るための別の仕掛けは要らなかった。

■見えないところ。 この検査が追えるのは、資源の名前が**文字列リテラル**である acquire だけである。変数に入った名前は静的には分からない（AIOS_STRICT_RESOURCE=1 でその場所を知らせる）。そこで、宣言された順序は**実行時にも効かせる**ことにした ―- 下位を持ったまま上位へ、の向きだけを許す。

```
acquire: dev is held, so bus must not be acquired now (declared order)
```

所見 6. 資源の順序は、**書かせるものではなく読み取るもの**だった。入れ子はプログラムの中に既にあり、それを集めれば順序の主張になる。そして閉路の検査は、**反例をそのまま報告できる**という副産物を持つ ―-「a の次に b を取っている場所」と「b の次に a を取っている場所」の両方が、辺の出どころとして残っているからである。

582 本の .aipl に対して、この検査で新たに落ちるものは **0 本**であった。

9.12　ノードをまたぐ待ち

レベルを「アクター型に載せる」べきだと第 3 版の初稿では書いたが、実際に穴を数えてみると**そこではなかった**。クラス注釈つきの引数は既に辺を張っており、注釈が無ければそもそも弾かれる。残っていた穴は**ノードをまたぐ待ち**だけであった。

宛先の実体は別ノードにあり、静的には見えない。そこで**ノード単位で階層化**する。

```
class Servo {
  var pos = 0;
  method step(d) : int @ 12 !{mut, log} { pos = pos + d; print("pos=" ++ pos); reply(pos); }
}
class Controller {
  method drive(d) : int @ 2 !{net} {          // 上位は低いレベル
    reply(now remote("rt0","servo1").step(d) timeout 200 else -1);
  }
}
node_level("rt0", 10);              // 実時間ノードは葉（高いレベル）
node_allow("rt0", "mut, log, time");
deploy("rt0", "Servo", "servo1");
```

```
[deploy] Servo -> rt0/servo1 (compiled at destination)
pos=3
drive = 3
```

そのノードへの待ちは必ず上へ向かうことを要求する。

```
[Type error] obligation level: Ctl2#go (@7) waits on node rt0 (floor @1);
             a wait across nodes must go up
```

配備の側でも照合する —- 下限より低いレベルのコードを置くと、そのノードへの待ちが上へ向かうという約束が破れる。

```
deploy: Servo.step is @3 but node rt0 has floor @10
```

効果を `node_allow` で国境で照合しているのと、まったく同じ形である。効果もレベルも、コードと一緒に運ばれ、受け入れ側の方針と突き合わされる。

注意 7. 併せて一つ直した。`any` がどの型とも単一化しなかったため、`now remote(n,a).m(x) timeout 100 else 0` が `type mismatch` になり**そもそも書けなかった**。`any` は「型が分からない」の印であって、合わない印ではない。563 本への影響は 0 本であった。

■到達点。 待ちの宛先がクラスとして分かるか、宛先ノードのレベルが宣言されている範囲では、デッドロック自由が型で保証される。注釈は要らない —- 推論が埋める。注釈を書けば、コードが持つ構造より**強い規律**を課せる。その外側は、期限が有限時間の失敗に変換する。資源の側も同様で、全体の順序を課さなければ取得順序のデッドロックが残る。レベルの推論（呼び出しグラフが非巡回なら自動で番号を振る）と、アクター型へのレベルの搭載が次の段である。

10　代表的なサンプルプログラム

最新仕様の代表として 10 本を挙げる。出力はすべて実行して得たものであり、**三つの処理系すべてで一致する**ことを確認している（§11）。

10.1　s01 —- アクターの基本

```
class Greeter {
  var count = 0;
  method init(name) { print("hello, " ++ name); }   // new のとき自動で呼ばれる
  method tick() : unit !{mut, log} {
    count = count + 1;
    print("tick " ++ count);
  }
}
var g = new Greeter("AIPL");
send g.tick();
send g.tick();
```

```
hello, AIPL
tick 1
tick 2
```

示すもの：クラス・フィールド・`init`・非同期送信・文字列連結。`++` が連結で `+` は数値専用である。**メリット**：状態を持つ並行部品が 10 行で書け、排他制御が要らない。**デメリット**：`send` は結果を返さないので、順序に依存する処理を書くと**送った順にしか保証がない**。

10.2　s02 —- 三種の送信

コードと出力は §3 に示した。**メリット**：待つ・待たないを**呼ぶ側**が選べる。同じメソッドを実時間側は `send` で、エージェント側は `future` で使える。**デメリット**：三種あるぶん、初学者はどれを使うか迷う。`now` を安易に選ぶと待ちが連鎖する。

10.3　s03 —- `reply` の規律

```
class Router {
  method route(k) : string {
    if (k > 0) { reply("positive"); } else { reply("non-positive"); }
  }
}
var r = new Router();
print(now r.route(5)  timeout 100 else "?");
print(now r.route(-1) timeout 100 else "?");
```

```
positive
non-positive
```

示すもの：戻り値型を宣言すると、全経路での `reply` が義務になる。`else` を消せば型エラーになる。**メリット**：「返信を忘れて相手が待ち続ける」が静的に止まる。**デメリット**：判定が構文の走査なので保守的である。`while` の本体にしか `reply` が無い場合、0 回実行されうるとみなして拒否する。

10.4　s04 —- 効果

コードは §5 に示した。

```
1
1
n=1
```

メリット：`!{}` と書けたメソッドは純粋だと分かるので、自由に複製・再配置できる。分散配置の判断が**型から機械的に決まる**。**デメリット**：語彙が 8 個に固定で、利用者が増やせない。また順序を持たないので「取得したら解放する」が書けない。

10.5　s05 —- 期限

コードと出力は §5 に示した。**メリット**：どんな相手でも待ちが有限で終わる。実時間側の設計が「最悪でも n ミリ秒」で組める。**デメリット**：`else` の値が**正常値と区別できない**。上の例の -1 は「失敗」を意味するが、型は `int` のままである。失敗を型で表すには直和型が要る。

10.6　s06 —- 実時間側の防壁

```
class Planner {                          // エージェント側
  method plan(g) : string !{ai, net} { reply(ai_call("plan " ++ g)); }
}
```

```
class Servo {                          // 実時間側。ai も net も持たない
  var p = new Planner();
  method step() : unit !{time, log} {
    send p.plan("grasp");              // 送るだけなら許される
    print("servo step");
  }
}
var s = new Servo();
send s.step();
```

```
servo step
```

示すもの：本稿の看板。send を now p.plan(...) に変えると、Servo.step が ai, net を宣言していないので型エラーになる。**メリット**：設計上の誤りが規約ではなく型検査で止まる。**デメリット**：この保証は**注釈を書いた場合にだけ**効く。注釈を省くと本体から推論されるので、誤りは誤りのまま通る。防壁は「宣言する」という利用者の行為に依存している。

10.7 s07 —- select

```
class Worker {
  method job(k) : string { reply("direct:" ++ k); }
  method serve() : unit !{log} {
    print("waiting");
    select {
      case job(k) -> { print("got " ++ k); reply("via select:" ++ k); }
      timeout 500 -> { print("timed out"); }
    }
  }
}
var w = new Worker();
send w.serve();
print(now w.job(1) timeout 800 else "?");
```

```
waiting
got 1
via select:1
```

示すもの：case 本体の reply は、囲む serve ではなく**受理した job への返信**になる。**メリット**：待ち受けと処理が一箇所に書ける。**デメリット**：where が無いので受理条件を状態で絞れない。また reply の帰属が直感に反しやすく、被覆検査もここで別扱いになる。

10.8 s08 —- 状態機械（become を使わずに書く）

```
class Door {
  var open = false;
  method state() : unit !{log} {
    if (open) { print("opened"); } else { print("closed"); }
  }
  method toggle() : unit !{mut, log} {
```

```
    if (open) { print("closing"); open = false; }
    else      { print("opening"); open = true; }
  }
}
var d = new Door();
send d.state();
send d.toggle();
send d.state();
```

```
closed
opening
opened
```

示すもの：§8 で become を外したあとの書き方。アクター型が実行時に変わらないので、外から見た型が嘘にならない。**メリット**：状態の数だけメソッドを増やさずに済む。**デメリット**：状態が多いと if が深くなる。状態遷移の構造は become 版のほうが読めた。表現力ではなく**読みやすさ**を手放した、という取引である。

10.9 s09 — アクターを引数に渡す

```
class Device { method ping() : unit !{log} { print("pong"); } }
class Driver {
  method attach(d: Device) : unit !{log} {   // Device 以外を渡すと型エラー
    print("attached");
    send d.ping();
  }
}
var dev = new Device();
var drv = new Driver();
send drv.attach(dev);
```

```
attached
pong
```

示すもの：引数に型注釈を書くとクラスが決まり、Device 以外を渡すと actor type mismatch になる。**メリット**：ドライバのような「相手を受け取って使う」部品が型安全に書ける。**デメリット**：注釈は**必須**である。省くと引数の型が決まらず、送信そのものが型エラーになる。また注釈は**クラス名**であってインタフェースではないので、「ping を持つ何か」を受け取ることはできない。

10.10 s10 — 三段のパイプライン

```
class Source { method get(i) : int !{} { reply(i * 10); } }
class Scale  {
  var src = new Source();
  method at(i) : int !{} { reply((now src.get(i) timeout 100 else 0) + 1); }
}
class Sink {
  var sc = new Scale();
  var total = 0;
```

```
  method run(n) : int !{mut, log} {
    var i = 0;
    while i < n do {
      total = total + (now sc.at(i) timeout 100 else 0);
      i = i + 1;
    }
    print("total=" ++ total);
    reply(total);
  }
}
var s = new Sink();
print("sum = " ++ (now s.run(4) timeout 1000 else -1));
```

```
total=64
sum = 64
```

示すもの：フィールドに new でアクターを持ち、now を二段重ねる実用的な形。**メリット**：段ごとに独立したアクターなので、後から段を差し替えたり分散配置したりできる。**デメリット**：now の連鎖は**待ちの連鎖**である。各段の期限が積み上がるので、外側の期限は内側の合計より大きくしなければならない。上の例で外側を timeout 100 にすると -1 になる。

10.11 s11 ── メッシュへの配備

コードと出力は §9.4 に示した。**メリット**：どのノードに何を置けるかが**型から決まる**。配置の判断が人手の規約でなくなる。**デメリット**：ノードの方針は文字列で書く（"mut, log, time"）。効果の語彙と綴りが一致しているかは実行時にしか分からない。

10.12 s12 ── 失敗を型で表す

コードと出力は §9.1 に示した。

```
fast ok 42
```

メリット：期限切れを正常値として流せない。is_ok を書かずに使うと型エラーになる。**デメリット**：待ちのたびに分岐が増える。else を書く従来の形も残してあるので、「必ず確かめる」を強制してはいない ── 選べるということは、選び損ねられるということでもある。

10.13 s13 ── 資源の取得と解放

コードと出力は §9.2 に示した。

```
read = 42
```

メリット：デバイスを掴んだまま抜ける経路が静的に消える。OS を書く目的に直結する。**デメリット**：資源の名前が**文字列リテラル**である。変数に入れた名前は追えない。資源のあいだの順序は s19 で入れた。

10.14 s14 ── 返信先の委譲

コードと出力は §9.3 に示した。

```
ans = 42
```

示すもの：ABCL/1 から落とした機構の回復。**メリット**：受付と処理を分けられる。受けたアクターが自分で答えずに、専門のアクターへ丸ごと渡せる。**デメリット**：線形性の検査は**メソッド内で閉じている**。渡した先が答えるかどうかは、渡した先の検査に委ねられる。各メソッドが局所的に規律を守ることで全体が成り立つ、という構成である。

10.15 s15 —- 義務レベル

コードと出力は §9.5 に示した。

```
level chain = 41
total = 41
```

示すもの：待ちが $1 \to 2 \to 3$ と一方向に流れる三段の連鎖。**メリット**：全メソッドに注釈が付けば、循環待ちが**構成的に**起きない。期限に頼らずに済む。**デメリット**：レベルはメソッド単位なので、同じクラスの別インスタンス同士の待ち（親が子を待つ木構造）も一律に弾かれる。注釈の付け忘れは推論が埋めるので弱点ではなくなったが、**注釈を書く意味**は残る —- コードが持つ構造より強い規律を先に宣言しておける。

10.16 s18 —- セッション型

コードと出力は §9.10 に示した。

```
pipeline = 41
```

示すもの：やり取りの順序を約束として書き、走らせる前に照合する。**メリット**：手順の取り違えが実行前に出る。実行時の検査と**同じ宣言**を使うので、二重に書く必要が無い。**デメリット**：静的に見えるのは、`protocol_start` と `protocol_end` が同じ並びにあるセッションだけ。アクターをまたぐ場合は実行時の検査に任せる。

10.17 s19 —- 資源への全体順序

コードと出力は §9.11 に示した。

```
w = 40
r = 21
```

示すもの：取得の入れ子から順序を読み取り、閉路が無いことを見る。**メリット**：注釈が要らない。逆順に取っている**二か所**が、そのままエラーに出る。**デメリット**：名前がリテラルの `acquire` しか追えない。そこだけは実行時の検査に任せる。

10.18 s17 —- `select` の規律

```
class Worker3 {
  method job(k) : string { reply("done:" ++ k); }
  method serve() : unit !{log} {
    print("waiting");
```

```
    select {
      case job(k) -> { print("got " ++ k); reply("via select:" ++ k); }
      timeout 500 -> { print("timed out"); }
    }
  }
}
var w = new Worker3();
send w.serve();
print(now w.job(7) timeout 800 else "?");
```

```
waiting
got 7
via select:7
```

示すもの：期限があり、待つメッセージ `job` を実際に誰かが送っている。どちらか欠けると警告が出る。**メリット**：`case` の綴り間違いと、書き忘れた送信が静的に出る。**デメリット**：送信が動的だと見えない。外部に公開しているプログラムでは検査そのものを行わない —- **言語が世界の全部を見ていない**ことの現れである。

10.19　s16 —- ノードをまたぐ義務レベル

コードと出力は §9.12 に示した。

```
pos=3
drive = 3
```

示すもの：レベルが**メッシュの国境を越える**。上位ノードは低いレベル、実時間ノードは高いレベル、という階層を宣言しておくと、「実時間側が上位を待つ」という逆流が型で止まる。**メリット**：分散デッドロックの主要な形（層をまたぐ待ちの循環）が消える。**デメリット**：ノード単位なので粗い。同じノードの中の待ちは、そのノードのレベル階層で別に見る必要がある。またレベルを宣言していないノードは検査されない —- 効果の方針と同じ弱点である。

10.20　19 本を通して見えること

	効いている仕組み	代価
s01–s03	型（`reply` の規律）	判定が保守的
s04, s06	効果	語彙が固定・注釈依存
s05, s10	期限	待ちが連鎖する
s07–s09	アクターの構造	受理条件・注釈・読みやすさ
s11	効果＋配備	方針が文字列
s12	失敗を型で表す	分岐が増える。従来の形も選べる
s13	順序つきの効果	名前が文字列。全体の順序が無い
s14	線形型	検査がメソッド内で閉じている
s15	義務レベル	粒度がメソッド単位
s16	ノード階層	粒度がノード単位。宣言しなければ検査されない
s17	select の規律	動的な送信・外部の送り手は見えない
s18	セッション型	同じ並びで完結するセッションに限る
s19	資源への全体順序	名前がリテラルの acquire に限る

第 2 版で「最も弱い」と書いた**失敗の扱い**（s05 の代価）は s12 で埋まり、「効果に順序が無い」は s13 で埋まった。代わりに新しい代価が四つ増えている。埋めた穴の数だけ、埋め方の癖が残るということである。

三つの仕組みはそれぞれ独立に理解できるが、**噛み合って初めて言えること**が s06 である。効果だけでも期限だけでも、あの誤りは止まらない。

11　三つの処理系と、揃っていることの確かめ方

AIPL には実装が三つある ―― 正となる OCaml 版（手書き 8,023 行）、Python の CLI 実装（15,101 行）、ブラウザで動く JavaScript 実装（6,331 行）である。第 2 版の時点からそれぞれ 700〜900 行増えているが、そのほとんどは**検査**であって機能ではない。三つ作る理由は二つある。JS 版はブラウザで動くので処理系を配らずに試してもらえること、そして**三つ揃えること自体が仕様の検査になる**ことである。

確かめ方は二本立てである。`run_all_impls.sh` が**通るプログラムの出力が一致するか**を見て、`run_reject_tests.sh` が**通ってはいけないプログラムが止まるか**を見る。後者は **30 本**の「弾かれるべきプログラム」からなる。

11.1　本稿の 10 本を三実装に通して見つかったもの

執筆のために 10 本を三実装へ通したところ、実装のバグが 5 件出た。

1. **フィールドを `new` で初期化するとアクターが動かない（OCaml 版）**。アクター生成のコードが二箇所に重複して書かれており、フィールドの初期化が `New` を扱えない評価器を呼んでいた。生成が例外で止まり、そのアクターはメッセージを一つも処理しないまま消えていた。s06 と s10 が丸ごと無出力だったのはこれである。
2. **同じ症状（JS 版）**。フィールドの初期化に `New` の枝が無く、フィールドがアクターではなく数値 `0`

になっていた。

3. **文法が三つ足りない（JS 版）**。`while ... do`、`become`、単項マイナスが文法に無かった。いずれも現行の文法要約に載っている構文である。
4. **スクリプト終了時に出力を取りこぼす（OCaml 版）**。20 回中 7 回、行が欠けていた。メールボックスが空になり、処理中のメッセージも無くなるまで待ってから抜けるようにした。
5. **`send` の効果の扱いが三者三様**。§5 で述べた規律に対し、Py 版と JS 版が引き継ぐ側になっていた。

所見 8. 5 件のうち 3 件が**正であるはずの OCaml 版**のものであった。「正」は仕様の基準という意味であって、実装が正しいという意味ではない。そして 1 と 2 は**同じ形のバグ**である —- 「アクター生成の本筋」と「フィールド初期化」が別の道を通っており、後者だけが `new` を扱えなかった。独立に書かれた三つの実装が同じ場所で同じ間違いをしたことは、これが**実装者の不注意ではなく設計の穴**であることを示している。フィールド初期化を式の評価と別扱いにしたこと自体が誤りだった。

11.2　現在の状態

サンプル 19 本のうち **17 本が三実装で出力一致**する（`s11` と `s16` は出力にノード名 `rt0/servo1` を含み、比較スクリプトが区切りに使う `/` で割れてしまうため自動比較から外した。内容は目視で一致を確認している）。ガイドの 8 本も一致する。弾かれるべき 30 本については、90 枠のうち **10 枠**が期待と食い違い、その内訳は JS 版 7・Py 版 3 である（**OCaml 版は 30 本すべてで期待どおりに振る舞う**）。残りはいずれも「OCaml 版にはあるが他の実装に無い検査」であり、言語仕様の穴ではなく追随の遅れである。

■第 3 版で三実装に展開したもの。 四つの機構はいずれも三実装すべてに入れた。その過程で、実装ごとに違う手当てが要った。

- **Py-I / JS-I**：メソッドが `reply` しないまま終わると、その時点で返信先が解決されてしまい、**委譲した先の `answer` が間に合わない**。`replyto` で持ち出した印を付けて自動解決を止めた。
- **JS-I**：`reply` は字句の段階でキーワードになるので、型注釈 `r: reply` が構文エラーになる。文法に別規則が要った。
- **Py-I / JS-I**：`remote("node","actor")` の構文が**そもそも無かった**。宛先を `"node/actor"` という名前へ落とす形で両方に足した。

12　考察

12.1　採否の判断は一貫していたか

振り返ると、落としたものには共通の理由がある —- **証明の形が変わるから**である。express mode は逐次性を、`where` は進行定理の枝を、返信先の一級性は「ちょうど一度」を、それぞれ壊す。交差型モデル検査と多相メソッドも、決定可能性と ρ の扱いという形で証明の側の理由で落ちている。

一方、採ったものには別の共通点がある —- **目的から要求されたから**である。三種の送信も、効果も、期限も、「分散 AI エージェントと実時間 OS を同じ言語で書き、混ぜない」という要求から出ている。

この二つの基準は、たまたま噛み合っている場面と、そうでない場面がある。噛み合っているのが効果と期限で、目的にも証明にも効いた。噛み合っていないのが返信先の一級性で、**目的からは欲しいのに証**

明の側から落とした。§6 で述べたとおり、これは線形型を入れれば取り戻せる。言い換えれば、AIPL の次の一歩は**証明の側の制約を緩める**方向にある。

12.2 「正」の実装という考え方の限界

§11 の所見は、本稿で最も実務的な発見である。三つの実装のうち一つを「正」と決め、他を追随させる運用をとってきた。これは仕様が一つに定まる利点がある一方で、**正の実装が間違えたとき、それを検出する仕掛けが無い**。

今回、効果伝播の穴が OCaml 版だけのものであったこと、フィールド初期化のバグが OCaml 版と JS 版に独立に存在したことは、どちらも「三つ並べて、通らないはずのものを通してみる」ことで初めて出た。出力の比較では出ない。正しいプログラムでは、どの実装も正しく動くからである。

ここから言えるのは、多実装を持つことの価値は**移植性ではなく、仕様の検査にある**ということである。そしてそのためには、正しいプログラムの集合と同じだけ、**誤ったプログラムの集合を資産として持つ**必要がある。

12.3 型・効果・期限という三点セットの評価

10 本を通して見ると、三つの仕組みの効き方には差がある。

型は最もよく効いている。`reply` の規律とアクター型の区別は、書き間違いの多くをその場で止める。代価は判定の保守性で、正しいのに拒否される場合がある。

効果は、効くときは非常に効くが**注釈依存**である。書かなければ推論されるだけで、誤りは通る。これは「効果を書かないと損をする」仕組みが無いためで、たとえば公開するアクターには注釈を義務づけるといった運用が要る。

期限は最も安上がりに強い性質を与える。構文を一つ足すだけで進行定理の枝が消える。しかし §5 で見たとおり、これは失敗を消すのではなく**失敗の形を変える**だけである。-1 が返ることと、正しく計算できたことを、型は区別しない。ここが現状で最も弱い箇所だと考えている。

12.4 三つの課題を片付けて分かったこと

第 2 版は残る課題を三つ挙げた —- 効果に順序、失敗を型で表す、返信先の一級化。第 3 版で三つとも片付けた。片付けて分かったことを書いておく。

■一つ目：どれも「型を厳しくする」話ではなかった。 三つとも、**既にある情報を捨てないようにする**話であった。失敗は実行時には分かっているのに型が捨てていた。資源の順序はプログラムに書いてあるのに検査が見ていなかった。返信先は実行時には値として存在するのに言語が隠していた。新しい理論を持ち込んだのではなく、**握り潰していたものを表に出した**のである。

■二つ目：待ちグラフは既にそこにあった。 循環待ちの検査は、新しい解析を書いたのではない。効果の伝播のために作っていた辺が、そのまま待ちグラフであった。**一つの目的で作った構造が別の目的に使える**のは、構造が現象に沿っているときに起きる。

■三つ目：それでもデッドロック自由は保証されない。 §9.5 の 10 行が示すとおりである。三つの課題は**どれもデッドロックの話ではなかった** —- 失敗を見えるようにし、資源の取り残しを止め、委譲を可能にしただけである。保証には義務レベルという別の道具が要る。課題を片付けることと、目標に近づくこ

とは同じではない。

12.5　機械検証との距離は広がった

正直に書いておくべきことがある。第 1 版の時点で、機械検証が済んでいるのは効果も期限も持たない中核 AIPL^{-} であり、効果と期限を加えた AIPL^{-2} は紙の上の証明であった。第 3 版では機構が四つ増えた。つまり**実装と機械検証の距離は、縮まるどころか広がっている**。

これは望ましくないが、避けがたくもある。機構を足すたびに形式化を追いかけると、「実装が何をしているか」を確かめる速度が落ちる。現状は、**機械検証の代わりに「弾かれるべきプログラム」30 本**が実装の規律を押さえている。これは証明ではないが、**三つの実装すべてに同じ問いを投げられる**という利点がある —- 証明は一つのモデルについてしか言えない。

12.6　残っている課題

1. **アクターをまたぐセッションの静的検査**。いまは実行時に任せている。
2. **名前が動的な `acquire`**。§9.11 の順序検査は文字列リテラルしか追えない。いまは実行時の検査で受けているが、資源を一級の値にして線形に扱えば静的に追える。
3. **メソッド内で起きた失敗の伝わり方**。宣言順序に反する `acquire` を実行時に捕まえたとき、呼び出し側の `now ... timeout ... else` が何を返すかが三実装で違う（それぞれ 停止 / `None` / `else` の値）。失敗を `result`$\langle\tau\rangle$ に載せる道と揃える必要がある。
4. AIPL^{-2} **以降の機械検証**。効果つき判断、期限、`future`$\langle\tau\,!\,\varepsilon\rangle$、そして第 3 版で足した四つ。
5. **実機のメッシュへの接続**。配備の輸送は現状ひとつのプロセス内で模している。Xinu ノード間の実際の転送に差し替える。

13　おわりに

本稿は AIPL の設計と実装を、採否の理由という軸でまとめ直した。ABCL/1 からは三種の送信・逐次性・返信を採り、express mode・返信先の一級性・`where` を落とした。小林の系譜からは効果と期限と線形性の考え方を採り、交差型モデル検査・多相メソッド・完全な所有権型を落とし、デッドロック自由な型は期限で代用した。どちらの源流にも無い `become` は、Akka Typed と同じ制限を課したうえで残した。

第 3 版では採否そのものを見直した。`become` を外し、失敗を型に出し、効果に順序を入れ、返信先を線形型で一級にし、メッシュ配備を足した。返信先の一級化は、**ABCL/1 から落とした機構を型の力で取り戻した**ことになる。

最後に、二つの版を通して得たことを記しておく。第 2 版では、サンプルを 10 本書いて三つの実装に通すだけで実装のバグが 5 件出た。うち 3 件は正であるはずの実装のものであった。第 3 版では、`become` を**外す**判断が、設計の議論ではなく **543 本を数えたこと**で決まった。

仕様は、書くだけでは検査されない。走らせて、複数の実装で走らせて、そして**実際に使われているかを数えて**、初めて検査される。

再現方法

```
# 本稿の14本を流す
```

```
cd ~/aios/abclcp
for f in docs/samples/paper/s*.aipl; do
  printf 'load %s\ncompile\n' "$PWD/$f" > /tmp/s.repl
  ./src/abclrepl_thread -q -f /tmp/s.repl
done

# 通るプログラムの一致 / 通ってはいけないプログラム21本の拒否（三実装）
sh tools/run_all_impls.sh
sh tools/run_reject_tests.sh

# 逃げ道（いずれも既定は厳しい側）
AIOS_LAX_WAIT=1     ... 循環待ちを警告に降格
AIOS_LAX_ACTOR=1    ... アクター型を区別しない
AIOS_LAX_ASSIGN=1   ... 未宣言の名前への代入を許す
AIOS_STRICT_DEADLINE=1 ... 期限のない待ちをエラーに昇格
```

参考文献

[1] Akinori Yonezawa, Jean-Pierre Briot, Etsuya Shibayama. Object-Oriented Concurrent Programming in ABCL/1. *OOPSLA 1986*, pp. 258–268.

[2] Akinori Yonezawa (ed.). *ABCL: An Object-Oriented Concurrent System*. MIT Press, 1990.

[3] Kenjiro Taura, Satoshi Matsuoka, Akinori Yonezawa. ABCL/f: A Future-Based Polymorphic Typed Concurrent Object-Oriented Language. 1994.

[4] Takuo Watanabe, Akinori Yonezawa. Reflection in an Object-Oriented Concurrent Language (ABCL/R). *OOPSLA 1988*.

[5] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

[6] Naoki Kobayashi, Akinori Yonezawa. Type-Theoretic Foundations for Concurrent Object-Oriented Programming. *OOPSLA 1994*.

[7] Naoki Kobayashi, Benjamin C. Pierce, David N. Turner. Linearity and the Pi-Calculus. *POPL 1996*.

[8] Naoki Kobayashi. A Partially Deadlock-Free Typed Process Calculus (1997) / A New Type System for Deadlock-Free Processes (*CONCUR 2006*).

[9] Naoki Kobayashi. A Generic Type System for the Pi-Calculus. *POPL 2001*.

[10] Naoki Kobayashi et al. Resource Usage Analysis. 2001/2002/2005.

[11] Naoki Kobayashi, C.-H. Luke Ong. A Type System Equivalent to the Modal Mu-Calculus Model Checking of Higher-Order Recursion Schemes. 2009.

[12] Naoki Kobayashi et al. Sized Types and Parallel Complexity.

[13] Lightbend. Akka Typed — `Behavior[T]` and behavior replacement.

[14] Ericsson. Erlang/OTP `gen_statem` Behaviour.

[15] 児玉靖司. AIPL: Actor based Intelligent Parallel Language — 設計から実現. 2026 年 8 月 21 日（第 1 版。形式的保証と穴の修正）.

[16] 児玉靖司. 同上 第 2 版 — ABCL/1 と小林の型システムから、何を採り何を落としたか. 2026 年 8 月 22 日.

[17] 児玉靖司. AIPL（ABCLc+）と ABCL/1 — 機能とモデルの観点からの比較. 2026 年 7 月 30 日.

[18] 児玉靖司. AIPL の言語仕様の検討 — ABCL/1 の機構の再導入と、分散 AI エージェント／実時間 OS という目的から. 2026 年 7 月 30 日.
[19] 児玉靖司. AIPL の型健全性と型安全性（第 2 版）. 2026 年 7 月 30 日.
[20] 児玉靖司. AIPL におけるメソッド戻り値型の推論 — reply の引数から型は決まるか. 2026 年 7 月 29 日.
[21] 児玉靖司. AIPL / ABCLc+ ユーザーズガイド. 2026 年 7 月 30 日.
[22] 児玉靖司. 小林研究の成果を AIPL に取り入れる場合の言語仕様 — 可能性、メリット、デメリット. 2026 年 7 月 30 日.